THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L08 Clustering I

Dr. Weikai Yang

Data Science and Analytics Thrust
& Computational Media and Arts Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

March 22, 2026

Course feedback

♡ Please scan the QR code and share your feedback.
No worries as it is anonymous. We sincerely appreciate your help.



Syllabus

Lec #	Topic	Lecturer
1	Introduction + course Info	Zixin
2-4	Supervised learning: regression and classification	Zixin
5	Feature engineering + Model evaluation and choice	Zixin
6	Model evaluation and choice + Feature selection	Zixin
7	**Ensemble learning (supervised learning)**	Weikai
	Midterm (28 March, 1:30 - 3:00 PM): mark it on calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	Dimension reduction	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	



Midterm Exam

- Time: Saturday, March 28, 1:30 - 3:00 PM
- Venue: Lecture Hall B
- Exam Coverage: L01 - L06
- Please check canvas for the reference to help you review and the sample questions
- No cheat sheets or unauthorized materials are allowed
- Bring your ID card and a calculator (basic non-programmable models only)



Ensemble Learning

- Aggregate multiple learners to obtain a better model
- The base learners should be accurate and diverse
- For a binary classification, if we have N **independent** base learners with error rate $\varepsilon < 0.5$, the error rate of their majority voting can be significantly decreased

$$\sum_{k=0}^{\lfloor N/2 \rfloor} \binom{N}{k} (1-\varepsilon)^k \varepsilon^{N-k} \leq \exp\left(-\frac{1}{2} N (1-2\varepsilon)^2\right)$$



Bagging

- steps
  - Draw B bootstrap samples from the training set
  - Train a base classifier $\hat{f}_1, \ldots, \hat{f}_B$ on each bootstrap sample
  - Aggregate the base classifiers to form a final classifier
- Reduce variance, but bias remain the same
  - Bias: systematic deviation from the target $\mathbb{E}[\hat{f}_i] - g = \mathbb{E}[\hat{f}] - g$
  - Variance: random fluctuations in the predictions $\mathbb{E}[(\hat{f}_i - \mathbb{E}[\hat{f}_i])^2] = \mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2]$
  - Expected error:

$$\mathbb{E}\left[\left(g - \hat{f}\right)^2\right] = \mathbb{E}\left[\left(g - \mathbb{E}\hat{f} + \mathbb{E}\hat{f} - \hat{f}\right)^2\right] = \underbrace{\left(g - \mathbb{E}\hat{f}\right)^2}_{\textbf{bias}} + \underbrace{\mathbb{E}\left[\left(\hat{f} - \mathbb{E}\hat{f}\right)^2\right]}_{\textbf{variance}}$$

- Should use low-bias learner for bagging
- Should use less correlated learners for bagging
  - Random forest: use random subset of features for each split



Gradient Boosting

- Core idea: build new models to correct mistakes of the current ensemble
- Each new model approximates the negative gradient of the loss
  - Under MSE loss, the negative gradient is exactly the residual: $r_i = y_i - \hat{y}_i$
- Procedure:
  1. Fit initial model $T^{(0)}$ on the training data
  2. Compute the gradient of the loss with respect to the predictions
  3. Fit a simple model $T^{(1)}$ to the gradient
  4. Update: $F \leftarrow F + \lambda T^{(1)}$
  5. Repeat steps 2-4 until the stopping condition met
- Boosting reduces bias (vs. bagging reduces variance)
- Can overfit $\Rightarrow$ tune: number of trees B, learning rate λ, tree depth d (usually use stumps, i.e., $d = 1$)
- Powers XGBoost, LightGBM



Adaptive Boosting (AdaBoost)

- Core idea: adaptively reweight samples — more weight on misclassified ones
- Procedure:
  1. Initialize equal weights $w_i^{(1)} = 1/N$
  2. For each round t: train base learner $T^{(t)}$ on weighted data $\{(w_1^{(t)}, x_1, y_1), \ldots, (w_N^{(t)}, x_N, y_N)\}$ and compute the weighted error rate $\varepsilon^{(t)}$.
  3. Compute learner weight: $\lambda^{(t)} = \frac{1}{2} \ln\left(\frac{1-\varepsilon^{(t)}}{\varepsilon^{(t)}}\right)$
  4. Update sample weights: $w_i^{(t+1)} \propto w_i^{(t)} \exp(-\lambda^{(t)} y_i T^{(t)}(x_i))$
- Final model: $F = \sum_t \lambda^{(t)} T^{(t)}$



Bagging vs. Boosting

	Bagging	**Boosting**
Training	Parallel	Sequential
Sampling	Bootstrap (with replacement)	Full data (reweighted)
Base learner	Low-bias, high-variance (e.g., deep trees)	Weak (e.g., stumps)
Diversity from	Different bootstrap samples (+random features in RF)	Correcting previous errors
Bias-variance	Reduces variance	Reduces bias
Overfitting	Resistant	Can overfit
Noise sensitivity	Robust	Sensitive
Examples	Bagging, Random Forest	Gradient Boosting, AdaBoost



Outline

1. What is clustering
2. Clustering metric
   - External clustering metric
   - Internal clustering metric
3. Hard Clustering Methods
   - Prototype-based Methods

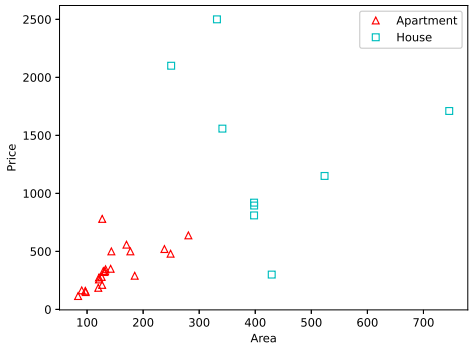


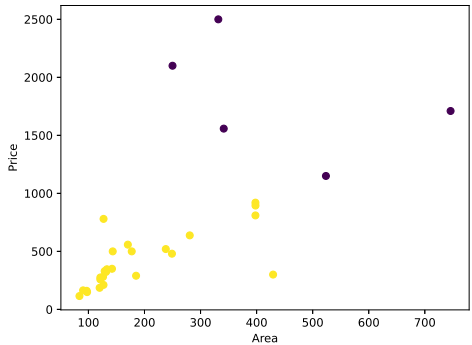
Clustering vs (house type) prediction

- Given a dataset containing n samples

$$\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \left(x^{(3)}, y^{(3)}\right), \ldots, \left(x^{(n)}, y^{(n)}\right)$$

- Task (vague): find interesting structures in the data

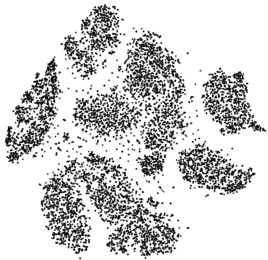


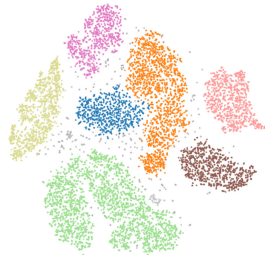




What is clustering

Finding groups in data!





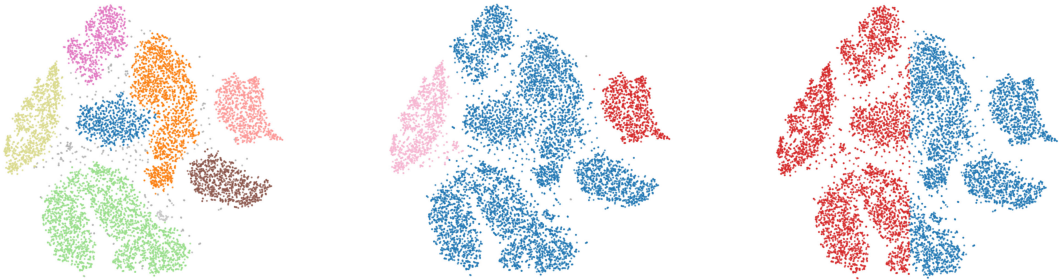
Mathematical definition

Finding groups in data!

- Given a dataset $\mathcal{D}$ consisting of n data points
- We want to separate them into K clusters (groups)
- Some key elements:
  - $\Delta = \{C_1, C_2, \ldots, C_K\}$: partition of $\mathcal{D}$
  - $\mathcal{L}(\Delta)$: cost (loss) of Δ



What is a good clustering result



- We need some numerical measures!
- Recap: we apply error metrics to measure performance of supervised learning algorithms.

⊙ **External**

⊙ **Internal**



What is a good clustering result

- **External**
  - Use external information, e.g., the category labels of images
  - Measure how well the clustering results match the ground-truth labels

  Q: What is the difference with classification? (Try to answer this question later)

- **Internal**
  - Do not use any external information
  - Assess the goodness of clusters based only on the intrinsic structure of the data
  - Focus on:
    - Compactness of clusters (how close points are within a cluster)
    - Separation between clusters (how distinct clusters are from each other)



Purity

- Measure the fraction of correctly classified points
- Formula:

$$\text{Purity} = \frac{1}{N} \sum_{i=1}^{k} \max_j |C_i \cap L_j|$$

where

- N: Total number of points
- k: Number of clusters
- C_i: Points in cluster i
- L_j: Points in ground truth class j
- $\max_j$: find the ground truth class that the cluster i has the most points in



Purity: example

- Assume that we have 30 samples, and the ground-truth labels $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

- Calculate the Purity for the following clustering results:

1. Clustering labels $c_8, \ldots, c_{17} = 1; c_{18}, \ldots, c_{27} = 2; c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$
2. Clustering labels $c_8, \ldots, c_{12} = 1; c_{13}, \ldots, c_{17} = 2; c_{18}, \ldots, c_{22} = 3; c_{23}, \ldots, c_{27} = 4; c_{28}, c_{29}, c_{30}, c_1, c_2 = 5, c_3, \ldots, c_7 = 6$
3. Each cluster contains 1 sample: $c_i = i, 1 \leq i \leq 30$;



Purity: example

- Assume that we have 30 samples, and the ground-truth labels $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

1. Clustering labels $c_8, \ldots, c_{17} = 1; c_{18}, \ldots, c_{27} = 2; c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$

$$(7 + 7 + 7)/30 = 0.7$$

2. Clustering labels $c_8, \ldots, c_{12} = 1; c_{13}, \ldots, c_{17} = 2; c_{18}, \ldots, c_{22} = 3; c_{23}, \ldots, c_{27} = 4; c_{28}, c_{29}, c_{30}, c_1, c_2 = 5, c_3, \ldots, c_7 = 6$

$$(3 + 5 + 3 + 5 + 3 + 5)/30 = 0.8$$

3. Each cluster contains 1 sample: $c_i = i, 1 \leq i \leq 30$;

$$\sum_{i=1}^{30} 1/30 = 1$$



Do we seek a cluster with higher or lower purity?

- Range of Purity is $[0, 1]$:

$$0 \leq \text{Purity} = \frac{1}{N} \sum_{i=1}^{k} \max_j |C_i \cap L_j| \leq \frac{1}{N} \sum_{i=1}^{k} |C_i| = 1$$

- Example: assume that we have 30 samples, and the ground-truth labels
  $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.
- Best clustering result should be
  $c_1, \ldots, c_{10} = 1$, $c_{11}, \ldots, c_{20} = 2$, $c_{21}, \ldots, c_{30} = 3$.
- Purity$= \sum_{i=1}^{3} 10/30 = 1$
- (Optional) Worst clustering result can be with purity=? Is it possible that purity$= 0$?



Purity

- Seek a cluster with **higher** ~~or lower~~ purity
- Example: assume that we have 30 samples, and the ground-truth labels

  $$\ell_1, \ldots, \ell_{10} = 1,\ \ell_{11}, \ldots, \ell_{20} = 2,\ \ell_{21}, \ldots, \ell_{30} = 3$$

- Best clustering result should be

  $$c_1, \ldots, c_{10} = 1,\ c_{11}, \ldots, c_{20} = 2,\ c_{21}, \ldots, c_{30} = 3$$

- Purity: $\sum_{i=1}^{3} 10/30 = 1$

- **Limitation**: Does not penalize over-clustering!
- Always correct but over-clustering: each cluster contains 1 sample

  $$c_i = i, 1 \leq i \leq 30$$

- Purity: $\sum_{i=1}^{30} 1/30 = 1$



Purity: demo

```
import numpy as np
from sklearn.metrics.cluster import contingency_matrix
def purity_score(y_true, y_pred):
    cm = contingency_matrix(y_true, y_pred)
    return np.sum(np.max(cm, axis=0)) / len(y_true)
y_gt = np.repeat([0,1,2],10)
y1 = np.repeat([0,1,2],10); y1 = np.concatenate([y1[3:], y1[:3]])
y2 = np.repeat([0,1,2,3,4,5],5); y2 = np.concatenate([y2[3:], y2[:3]])
y3 = np.array(range(30))
print(purity_score(y_gt, y1)) # 0.7
print(purity_score(y_gt, y2)) # 0.8
print(purity_score(y_gt, y3)) # 1.0
```



Rand Index (RI)

- Measure the fraction of **correct pairs** between the clustering labels and the ground truth labels
- For a sample pair (x_i, x_j), consider their clustering labels c_i, c_j and groundtruth labels l_i, l_j
- If $c_i = c_j$ but $l_i \neq l_j$, is it TP/TN/FP/FN?
- Rand index: examine all the sample pairs, inspired by binary classification
  - True Positive (TP): $c_i = c_j$, $l_i = l_j$
  - True Negative (TN): $c_i \neq c_j$, $l_i \neq l_j$
  - False Positive (FP): $c_i = c_j$, $l_i \neq l_j$
  - False Negative (FN): $c_i \neq c_j$, $l_i = l_j$

$$\mathrm{RI} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}} = \frac{\mathrm{TP} + \mathrm{TN}}{\binom{n}{2}}$$

- **Range**: 0 to 1 (1 = perfect match)
- Why: 1 = perfect match? Perfect match $\iff \mathrm{FN} = \mathrm{FP} = 0$



Rand Index (RI): example

- Assume that we have 30 samples, and $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

- Calculate the RI for the following clustering results:

1. $c_8, \ldots, c_{17} = 1; c_{18}, \ldots, c_{27} = 2; c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$
2. A cluster contains 29 samples and the other contains 1 sample: $c_1, \ldots, c_{29} = 1, c_{30} = 2$
3. Each cluster contains 1 sample: $c_i = i, 1 \leq i \leq 30$



Rand Index (RI): example

- Assume that we have 30 samples, and $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

1. $c_8, \ldots, c_{17} = 1$; $c_{18}, \ldots, c_{27} = 2$; $c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$
- $TP = 3 \times \left[\binom{7}{2} + \binom{3}{2}\right] = 72$
- $TN = 3 \times (7 \times 17 + 3 \times 13)/2 = 237$
- $RI = (72 + 237)/(30 \times 29/2) = 0.710$



Rand Index (RI): example

- Assume that we have 30 samples, and $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

2. A cluster contains 29 samples and the other contains 1 sample: $c_1, \ldots, c_{29} = 1, c_{30} = 2$
- $TP = \binom{10}{2} + \binom{10}{2} + \binom{9}{2} = 126$
- $TN = 20$
- $RI = (126 + 20)/(30 \times 29/2) = 0.336$



Rand Index (RI): example

- Assume that we have 30 samples, and $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

3. Each cluster contains 1 sample: $c_i = i, 1 \leq i \leq 30$
- $TP = 0$
- $TN = 20 \times 30/2 = 300$
- $RI = (300)/(30 \times 29/2) = 0.690$



Rand Index (RI): pros & cons

- Seek a cluster with high RI!
- Range: $[0, 1]$

- **Pro**: Punish over-clustering (still remember 'Purity'?)
- **Con/limitation**: Does not consider the possibility of chance agreements between the two clusters!

Q: What is chance agreement? How to solve the issue?

$\Rightarrow$ Adjusted rand index (ARI)



Adjusted rand index (ARI)

- **Motivation:** adjusted for chance to handle random clustering
- **How:** using the expected similarity of all pair-wise comparisons between clusterings specified by a random model

$$\text{ARI} = \frac{\text{RI} - \text{Expected RI}}{\max(\text{RI}) - \text{Expected RI}}$$

- **Range**: -1 to 1 ($1 =$ perfect match).
- Why: $1 =$ perfect match?



Adjusted rand index (ARI) (optional)

$$ARI = \frac{\sum_{i,j}\binom{n_{ij}}{2} - \left(\sum_i \binom{n_{i+}}{2}\sum_j \binom{n_{+j}}{2}\right)/\binom{n}{2}}{\frac{1}{2}\left(\sum_i \binom{n_{i+}}{2} + \sum_j \binom{n_{+j}}{2}\right) - \left(\sum_i \binom{n_{i+}}{2}\sum_j \binom{n_{+j}}{2}\right)/\binom{n}{2}}$$

- contingency table

	c_1	c_2	$\cdots$	c_k	Sum
l_1	n_{11}	n_{12}	$\cdots$	n_{1k}	n_{1+}
l_2	n_{21}	n_{22}	$\cdots$	n_{2k}	n_{2+}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
l_m	n_{m1}	n_{m2}	$\cdots$	n_{mk}	n_{m+}
Sum	n_{+1}	n_{+2}	$\cdots$	n_{+k}	n

Reference:

1. Understanding the Adjusted Rand Index and Other Partition Comparison Indices Based on Counting Object Pairs[Link]
2. Comparing partitions [Link]

Adjusted Rand Index (ARI): demo

```
import numpy as np
from sklearn.metrics.cluster import rand_score, adjusted_rand_score
y_gt = np.repeat([0,1,2],10)
y1 = np.repeat([0,1,2],10); y1 = np.concatenate([y1[3:], y1[:3]])
y2 = np.array([0]*29 + [1])
y3 = np.array(range(30))
print(rand_score(y_gt, y_gt), adjusted_rand_score(y_gt, y_gt)) #1.0 1.0
print(rand_score(y_gt, y1), adjusted_rand_score(y_gt, y1))#
    0.7103448275862069 0.3233333333333333
print(rand_score(y_gt, y2), adjusted_rand_score(y_gt, y2)) #
    0.335632183908046 0.0
print(rand_score(y_gt, y3), adjusted_rand_score(y_gt, y3)) #
    0.6896551724137931 0.0
```

Try to implement it yourself and compare the result with sklearn!



Fowlkes-Mallows Index (FMI)

- Measures the geometric mean of precision and recall for pairs of points.
- Again, 'inspired by' binary classification
- Formula:

$$\text{FMI} = \sqrt{\text{Precision} \cdot \text{Recall}}$$

where:

  - Precision: fraction of pairs correctly clustered together $\frac{\text{TP}}{\text{TP} + \text{FP}}$
  - Recall: fraction of true pairs correctly identified $\frac{\text{TP}}{\text{TP} + \text{FN}}$

- **Range**: 0 to 1 (1 = perfect match)
- Why: 1 = perfect match?



Useful facts: entropy of a random variable

- Quantifies the average level of uncertainty or information

$$H(X) = \mathbb{E}[-\ln P(X)] = \sum_{x \in X} -P(x) \ln P(x)$$

- Some examples
  - $P(X = a) = 1, H(X) = -1 \cdot \ln(1) = 0$
  - $P(X = a) = 2/3, P(X = b) = 1/3, H(X) = -2/3 \ln(2/3) - 1/3 \ln(1/3) \approx 0.6365$
  - $P(X = a) = P(X = b) = 1/2, H(X) = -1/2 \ln(1/2) - 1/2 \ln(1/2) = \ln 2 \approx 0.6931$
  - $P(X = a) = P(X = b) = P(X = c) = 1/3, H(X) = \ln 3 \approx 1.0986$



Useful facts: conditional entropy

- Quantifies the amount of information needed to describe Y given the value of X

$$H(Y|X) = \mathbb{E}[-\ln P(Y|X)] = \sum_{x\in X, y\in Y} -P(x,y)\ln P(y|x)$$
$$= \sum_{x\in X, y\in Y} -P(x,y)\ln \frac{P(x,y)}{P(x)}$$

  - Or you can view it like this

$$H(Y|X) = \sum_{x\in X} P(X=x)H(Y|X=x) = \sum_{x\in X} P(x)\sum_{y\in Y} -P(y|x)\ln P(y|x)$$
$$= -\sum_{x\in X, y\in Y} P(x)P(y|x)\ln P(y|x)$$

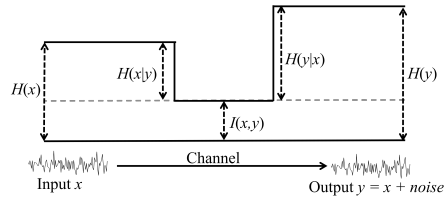
- $0 \le H(Y|X) \le H(Y)$, $H(Y|X) = H(X,Y) - H(X)$
- $H(Y|X) = 0$ if Y is determined by X
- $H(Y|X) = H(Y)$ if X and Y are independent

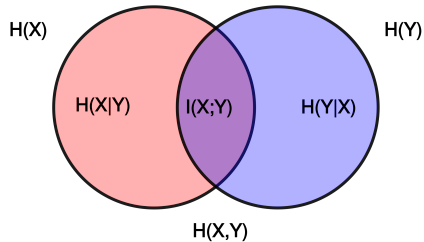


Useful facts: mutual information

- Quantifies the amount of information obtained about Y after observing X (and vice versa)

$$\begin{aligned} I(X,Y) = I(Y,X) &= H(X) - H(X|Y) \\ &= H(Y) - H(Y|X) \\ &= H(X) + H(Y) - H(X,Y) \\ &= H(X,Y) - H(X|Y) - H(Y|X) \end{aligned}$$





(Source: Wikipedia)

- Let X be the clustering results

  Y be the groundtruth labels

  then MI reflects the alignment between them



Normalized Mutual Information (NMI)

- Normalized to account for different numbers of clusters.
- Formula:

$$\text{NMI} = \frac{2 \cdot I(C, L)}{H(C) + H(L)}$$

  where:
  - $I(C; L)$: Mutual information (MI) between clusters C and labels L.
  - $H(C)$, $H(L)$: Entropy of clusters and labels.
- **Range**: 0 to 1 (1 = perfect match)
- Why: 1 = perfect match?



Normalized Mutual Information (NMI): example

- Assume that we have 30 samples, and the ground-truth labels $\ell_1, \ldots, \ell_{10} = 1$, $\ell_{11}, \ldots, \ell_{20} = 2$, $\ell_{21}, \ldots, \ell_{30} = 3$.

- Calculate MI and NMI for the following clustering results

1. $c_1, \ldots, c_{10} = 1$, $c_{11}, \ldots, c_{20} = 2$, $c_{21}, \ldots, c_{30} = 3$
2. $c_8, \ldots, c_{17} = 1$; $c_{18}, \ldots, c_{27} = 2$; $c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$
3. $c_8, \ldots, c_{12} = 1$; $c_{13}, \ldots, c_{17} = 2$; $c_{18}, \ldots, c_{22} = 3$; $c_{23}, \ldots, c_{27} = 4$; $c_{28}, c_{29}, c_{30}, c_1, c_2 = 5, c_3, \ldots, c_7 = 6$
4. Each cluster contains 1 sample: $c_i = i, 1 \leq i \leq 30$;



Normalized Mutual Information (NMI): example

1. $c_1, \ldots, c_{10} = 1$, $c_{11}, \ldots, c_{20} = 2$, $c_{21}, \ldots, c_{30} = 3$

$$MI = H(C) - H(C|L) = H(C) = -\sum_{c=1}^{3} P(C = c) \ln P(C = c) = \ln 3; NMI = 1$$

2. $c_8, \ldots, c_{17} = 1; c_{18}, \ldots, c_{27} = 2; c_1, \ldots, c_7, c_{28}, c_{29}, c_{30} = 3$

$$\begin{aligned} MI = H(C) - H(C|L) &= \ln 3 - \sum_{l=1}^{3} P(L = l) H(C|L = l) \\ &= \ln 3 + \sum_{l=1}^{3} \frac{1}{3} \left(\frac{7}{10} \ln \frac{7}{10} + \frac{3}{10} \ln \frac{3}{10} \right) \approx 0.4877 \end{aligned}$$

$$NMI = 2MI/2 \ln 3 \approx 0.4440$$



Normalized Mutual Information (NMI): example

3. $c_8, \ldots, c_{12} = 1; c_{13}, \ldots, c_{17} = 2; c_{18}, \ldots, c_{22} = 3; c_{23}, \ldots, c_{27} = 4; c_{28}, c_{29}, c_{30}, c_1, c_2 = 5, c_3, \ldots, c_7 = 6$

$$MI = H(C) - H(C|L) = \ln 6 + \left(\frac{2}{10}\ln\frac{2}{10} + \frac{5}{10}\ln\frac{5}{10} + \frac{3}{10}\ln\frac{3}{10}\right) \approx 0.7621$$

$$MI = H(L) - H(L|C) = \ln 3 + \frac{3}{6} \times \left(\frac{2}{5}\ln\frac{2}{5} + \frac{3}{5}\ln\frac{3}{5}\right) + \frac{3}{6} \times 0 \approx 0.7621$$

$$NMI = 2MI/(H(C) + H(L)) \approx 0.5273$$

4. $c_i = i, 1 \leq i \leq 30$;

$$MI = H(L) - H(L|C) = \ln 3 - 0 = \ln 3$$

$$NMI = 2\ln 3/(\ln 3 + \ln 30) \approx 0.4883$$



Normalized Mutual Information (NMI): demo

```
import numpy as np
from sklearn.metrics.cluster import mutual_info_score,
    normalized_mutual_info_score
y_gt = np.repeat([0,1,2],10)
y1 = np.repeat([0,1,2],10); y1 = np.concatenate([y1[3:], y1[:3]])
y2 = np.repeat([0,1,2,3,4,5],5); y2 = np.concatenate([y2[3:], y2[:3]])
y3 = np.array(range(30))
print(mutual_info_score(y_gt, y_gt), normalized_mutual_info_score(y_gt,
    y_gt))
print(mutual_info_score(y_gt, y1), normalized_mutual_info_score(y_gt, y1))
print(mutual_info_score(y_gt, y2), normalized_mutual_info_score(y_gt, y2))
print(mutual_info_score(y_gt, y3), normalized_mutual_info_score(y_gt, y3))
```



V-measure

- A harmonic mean of two complementary components:
  - **Homogeneity**: Measures how pure each cluster is (all points in a cluster belong to a single class).
  - **Completeness**: Measures how well all points of a class are assigned to the same cluster.

$$\text{Homogeneity} = 1 - \frac{H(L|C)}{H(L)}, \quad \text{Completeness} = 1 - \frac{H(C|L)}{H(C)}$$

where:

- $H(C|L)$: Conditional entropy of clusters given labels.
- $H(L|C)$: Conditional entropy of labels given clusters.
- $H(C)$, $H(L)$: Entropy of clusters and labels.

- Formula:

$$\text{V-measure} = 2 \cdot \frac{\text{Completeness} \cdot \text{Homogeneity}}{\text{Homogeneity} + \text{Completeness}}$$

- **Range**: 0 to 1 (1 = perfect match).
- Why: 1 = perfect match?



External metrics: checklist

- Using ground truth labels $\Longrightarrow$ supervised learning?

- Key metrics:
  - **Purity:** measure the fraction of correctly classified points.

    (Start from confusion matrix:)
  - **Adjusted Rand Index (ARI):** compare cluster assignments to true labels, adjusted for chance.
  - **Fowlkes-Mallows Index (FMI):** geometric mean of precision and recall for pairs of points.

    (Start from entropy:)
  - **Normalized Mutual Information (NMI):** measure mutual information between cluster assignments and true labels.
  - **V-measure**: harmonic mean of homogeneity and completeness.



External metrics: checklist

- Understand each metric:
  - Formula?
  - Pros and cons?
  - Range? Seek a clustering with higher/lower value of the metric?
- Choose the metric based on the specific use case and requirements



Internal clustering metric

- Silhouette Coefficient
- Calinski-Harabasz Index (Variance Ratio Criterion)
- Davies-Bouldin Index
- Dunn index



Silhouette Score

- Measures how similar a point is to its own cluster compared to other clusters
- Formula for a single point:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} = \begin{cases} 1 - a(i)/b(i) & a(i) < b(i) \\ a(i)/b(i) - 1 & a(i) \geq b(i) \end{cases} \in [-1, 1]$$

  where:

  - $a(i)$: Average distance of point i to other points in the same cluster.
  - $b(i)$: Average distance of point i to points in the nearest neighboring cluster.
- Overall score is the average of $s(i)$ for all points.
- **Range**: -1 to 1 (1 = well-clustered, -1 = poorly clustered).
  - Why?



Calinski-Harabasz Index

- Measures the ratio of between-cluster dispersion to within-cluster dispersion
- Formula:

$$\text{CH} = \frac{\text{Between-Cluster Dispersion}/(k-1)}{\text{Within-Cluster Dispersion}/(N-k)}$$

  where:

  - N: Total number of points.
  - k: Number of clusters.
- Between-Cluster Dispersion: Sum of squared distances between cluster centers and global center, i.e., $\sum_{i=1}^{k} n_i \|c_i - c\|^2$
  - Within-Cluster Dispersion: Sum of squared distances between points and their cluster center. $\sum_{i=1}^{k} \sum_{x \in C_i} \|c_i - x\|^2$
- **Higher values** indicate better clustering
  - Why?
  - (Optional) Similar to one-way ANOVA F-test statistic



Davies-Bouldin Index

- Measures the average similarity ratio of within-cluster scatter to between-cluster separation
- Formula:

$$\text{DB} = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{S_i + S_j}{d(c_i, c_j)} \right)$$

where:

- k: Number of clusters.
- S_i: Average distance of points in cluster i to its center.
- $d(c_i, c_j)$: Distance between centers of clusters i and j.
- **Lower values** indicate better clustering
- Why?



Dunn Index

- Measures the ratio of the smallest inter-cluster distance to the largest intra-cluster distance
- Formula:

$$\text{Dunn Index} = \frac{\min_{1 \le i < j \le k} d(C_i, C_j)}{\max_{1 \le l \le k} \text{diam}(C_l)}$$

where:

- $d(C_i, C_j)$: Distance between clusters C_i and C_j (e.g., minimum or average distance).
- $\text{diam}(C_l)$: Diameter of cluster C_l (e.g., maximum distance between points in the cluster).
- k: Number of clusters.
- **Higher values** indicate better clustering:
- Large inter-cluster distances (well-separated clusters).
- Small intra-cluster distances (compact clusters).
- Sensitive to noise and outliers (Why)?



Internal metrics: checklist

- Evaluate clustering quality without ground truth $\Longrightarrow$ unsupervised learning!
- Understand each metric:
  - Formula?
  - Pros and cons?
  - Range? Seek a clustering with higher/lower value of the metric?

- Choose the metric based on the clustering goals and data characteristics



Internal metrics: checklist

- Key metrics:
  - **Silhouette Score:** measure cohesion and separation.
  - **Calinski-Harabasz Index:** measure dispersion ratio.
  - **Davies-Bouldin Index:** measure average cluster similarity.
  - **Dunn Index:** measure the ratio of inter-cluster distance to intra-cluster diameter.

- Each metric has its strengths and is suited for different scenarios:
  - **Silhouette Score:** general-purpose, easy to interpret.
  - **Calinski-Harabasz Index:** emphasizes dispersion and separation.
  - **Davies-Bouldin Index:** focuses on cluster similarity.
  - **Dunn Index:** highlights compactness and separation explicitly.



Question about clustering

- What is the difference between classification and clustering?

- Why do we need so many different metrics?



Clustering methods

- Can we find good clustering results by directly optimizing the above metrics?
  - **Non-Differentiable:** Most metrics are discrete or non-smooth, making gradient-based optimization infeasible.
  - **Combinatorial Complexity:** Many require evaluating all possible cluster assignments, which is computationally intractable.
- **Clustering Goal:** Find natural groupings in data without predefined labels, unlike supervised learning.

- We use heuristic algorithms that optimize proxy objectives instead.
- Metrics serve as evaluator and assess quality after clustering, not during optimization.



Clustering methods

- **Hard** partitioning clustering
  - A data point belongs to only one cluster

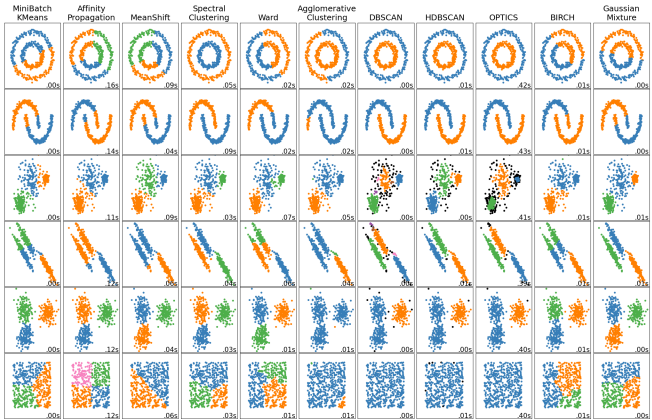
- **Soft** partitioning clustering
  - γ_{ki} is the degree of membership of point i to cluster k
  - $\sum_k \gamma_{ki} = 1, \forall i$

- Recap: hard and soft classification



Clustering methods

- We will only cover some of them, for more please visit sklearn



Popular hard partitioning clustering methods

- Prototype-based methods
  - e.g., k-means, k-medoids
- Density-based methods
  - e.g., DBSCAN
- Hierarchical methods
  - ⋆ Agglomerative/divisive
  - HDBSCAN



Prototype-based clustering methods

- **Definition**: Clustering methods that represent each cluster by a prototype (e.g., centroid, medoid) summarizing the group
- **Key Idea**: Assign data points to the nearest prototype, optimizing a distance-based objective (e.g., sum of squares)

- **Examples**:
  - k-Means: use centroids (mean of points) to minimize within-cluster variance.
  - k-Medoids: use actual data points (medoids) for robustness to outliers

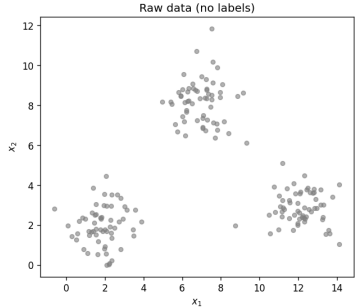
- **Pros**: Simple, scalable, and effective for spherical clusters.
- **Cons**: Sensitive to initialization and cluster shape.

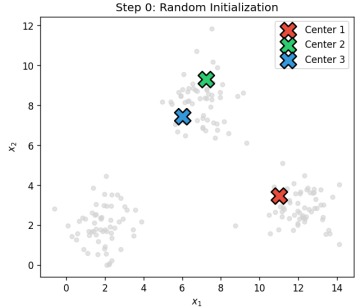


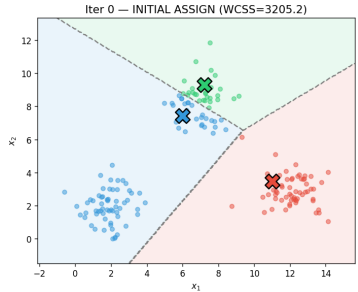
Kmeans

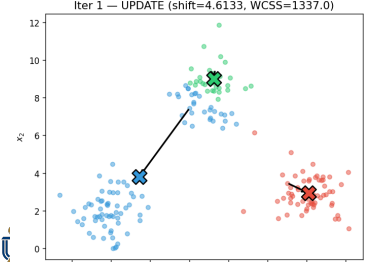


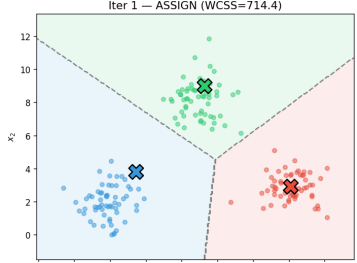
kmeans at work

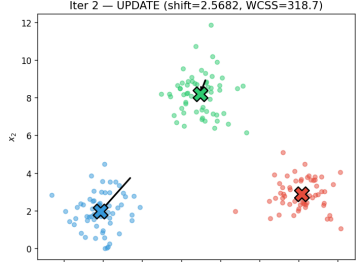












kmeans: procedure

Algorithm 1: k-means Clustering

Require: Dataset $X = \{x_1, x_2, \ldots, x_n\}$, number of clusters K

Ensure: Cluster assignments for each x_i

1: Initialize centers $\{\mu_1, \mu_2, \ldots, \mu_K\}$ (e.g., random points from X)

2: **repeat**

3: **Assign step**: Assign each point x_i to the closest center

$$r_i = \arg \min_{k \in \{1,\ldots,K\}} \|x_i - \mu_k\|$$

4: **Update step**: Recompute each center as the mean of all points assigned to it

$$\mu_k = \frac{\sum_{i:r_i=k} x_i}{\sum_{i:r_i=k} 1}$$

5: **until** Stop conditions are met



kmeans: stop conditions

- Maximum iterations reached
- Assignments no longer change
- Convergence of cost $\mathcal{L}$ to a local optimum (defined next $\Rightarrow$)



kmeans: cost function

$$\text{Within-Cluster Sum of Squares (WCSS)} \qquad \mathcal{L}(\Delta) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

- K-means solves a least-squares problem
- the cost $\mathcal{L}$ is called quadratic distortion

Proposition. The K-means algorithm decreases $\mathcal{L}(\Delta)$ at every step.
Sketch of proof

- reassigning the labels can only decrease $\mathcal{L}$
- reassigning the centers μ_k can only decrease $\mathcal{L}$



kmeans: optimal number of clusters

- Elbow method: A heuristic method to determine the optimal number of clusters k.
- **Steps:**
  1. Run k-means clustering for a range of k values (e.g., 1 to 10).
  2. Calculate the Within-Cluster Sum of Squares (WCSS) for each k.
  3. Plot WCSS against k.
  4. Look for an "elbow" point where adding more clusters yields diminishing returns.
- **Goal:** Balance between minimizing WCSS and avoiding overfitting with too many clusters.
- **Is there other ways to determine the optimal number of clusters?**

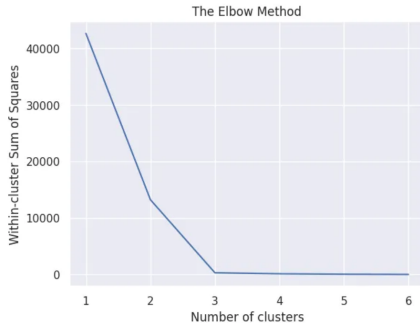


Figure: Elbow plot (illustration)



kmeans: Pros & Cons

- **Pros:**
  - Simple and fast.
  - Works well for spherical clusters.
- **Cons:**
  - Sensitive to initial centers.
  - Assumes clusters of similar size and density.



How to improve k-means?

- By improving the initialization of the centers



Initialization of the centers

- kmeans: starts with K **data points** at random

  Q: What's wrong with choosing K data points at random?
- **A:** The probability of hitting all K clusters with K samples approaches 0 when K is large

Solutions:

- Fastest First Traversal (greedy simple approach to spread out centers)
- k-means++ (randomized, theoretically backed approach to spread out centers)
- "K-log K" Initialization (start with enough centers to hit all clusters, then prune down to K)



Fastest First Traversal (FFT)

1. **Procedure:**

1. **Input** dataset X with $|X| = n$
2. **Output** $C \in X$ with $|C| = k \leq n$
3. While $|C| < k$
   - Find $T(t) = \min_{x \in C} d(t, x)$
   - Find $x_* = \arg\max_t T(t)$
   - $C = C \cup \{x_*\}$
4. **Return** C



k-Means++: smart initialization for k-Means

- **Procedure:**

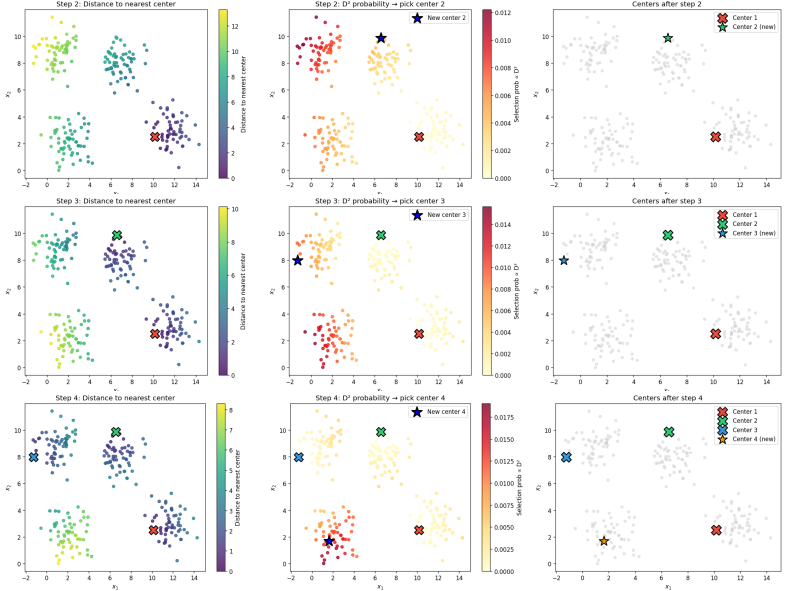
1. Choose the first center randomly from the data points.
2. For each remaining point, compute its squared distance to the nearest existing center.
3. Select the next center with probability proportional to that squared distance.
4. Repeat until K centers are chosen.

- **Key idea:** Spreads centers by favoring points farther from existing ones, balancing randomness and determinism.

- **Advantage:** Reduces sensitivity to poor initialization, theoretically guarantees $O(\log K)$-competitive solution.



k-Means++: step-by-step illustration



k-Means++: smart initialization for k-Means (optional)

- **Bound:** $E[\phi_{\text{K-Means++}}] \leq 8(\ln K + 2) \cdot \phi_{\text{OPT}}$

- **Proof sketch:**

1. $8(\log K + 2) = O(\log K)$ in big-O terms.
2. Log factor from sequential covering over K steps.

- Reference: D Arthur, S Vassilvitskii. k-means++: The advantages of careful seeding [Link]



"K-log K" Initialization

- **Procedure:**
- Start with enough centers to hit all clusters, then prune down to K

Q: How to prune?

A: Minimize the loss!

$$\mathcal{L}(\Delta) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

- Internal ~~external~~ clustering metrics



k-medoids



k-medoids

- Similar to k-means but use actual data points (medoids) as cluster centers.

- **Procedure:**

1. Initialize k medoids randomly
2. Assign each point to the nearest medoid
3. Update medoids to minimize the total cost
4. Repeat until convergence



k-medoids: pros & cons

- **Pros:**
  - Robust to noise and outliers.
  - Works with any distance metric.
- **Cons:**
  - Computationally expensive for large datasets



k-medoids

- Cost $\mathcal{L}(\Delta) = \sum_k \sum_{i \in C_k} \|x_i - \mu_k\|$ with $\mu_k \in \mathcal{D}$
- Exercise: Show that in 1D $\arg\min_\mu \sum_i |x_i - \mu|$ is the median of $\{x_i\}$

- Optimizing this cost is *NP*-hard
- So it has attracted a lot of interest in theoretical computer science (generally called "Facility Location")



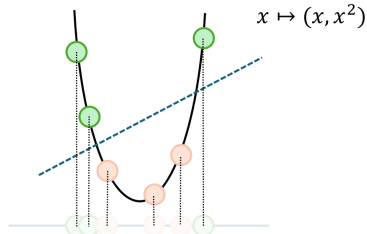
Handle non-linear data

- Many real-world datasets are not linearly separable in their original space
- **Possible solution:** They might be linearly separable after mapping to higher-dimensional space $x \mapsto \phi(x)$

Linearly Non-separable



Linearly Separable



- **Con:** it brings higher computational complexity when calculating the mapping results and solving it $\Longrightarrow$ **Solution?**



Kernel tricks

- In many cases, we don't need to compute $\phi(x)$ but only $\phi(x)^\top \phi(y)$
  - In SVM, we have

$$\mathcal{L}(w, b, \lambda) = \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j (x_i^\top x_j)$$

- This can sometimes be computed much more efficiently
- $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \phi(\mathbf{x}) = \begin{pmatrix} x_1 x_1 \\ x_1 x_2 \\ x_2 x_1 \\ x_2 x_2 \end{pmatrix}$
- What is the complexity of computing $\phi(x)$?
- What is the complexity of directly computing $\phi(x)^\top \phi(y)$?
- $\phi(x)^\top \phi(y) = (x^\top y)^2 = K(x, y)$



Common kernel functions

- **Linear Kernel**: $K(x, y) = x^\top y$ (no transformation)
- **Polynomial Kernel**: $K(x, y) = (x^\top y + c)^d$ (degree d, constant c)
- **Radial Basis Function (RBF) Kernel**: $K(x, y) = \exp(-\gamma \|x - y\|^2)$ (parameter γ)
- **Sigmoid Kernel**: $K(x, y) = \tanh(\alpha x^\top y + c)$

Note: Choice of kernel depends on the data and problem



Kernel SVM: case of XOR problem

Problem:

- Data: XOR points in 2D:
  $(0,0), (1,1) =$ Class 0
  $(0,1), (1,0) =$ Class 1
- Not linearly separable in 2D
- Familiar?

Solution:

- Use a polynomial kernel:
  $K(x,y) = (x^\top y)^2$
- High degree $\Rightarrow$ better?
- Maps to 3D: $\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$
- (0,1) $\mapsto$ (0,0,1); (1,0) $\mapsto$ (1,0,0)
- (1,1) $\mapsto$ (1,$\sqrt{2}$,1); (0,0) $\mapsto$ (0,0,0)
- Now linearly separable in 3D!



Kernel SVM: case of RBF Kernel

Problem:

- Inner points: Class 0;
- Outer points: Class 1.

- Not linearly separable in 2D

Solution:

RBF Kernel:
$K(x, y) = \exp(-\gamma \|x - y\|^2)$

- Maps to an infinite-dimensional space: (`https://en.wikipedia.org/wiki/Radial_basis_function_kernel`)
- Measures similarity based on distance.
- γ controls the "reach" of the kernel.
- The decision function will be $\sum_i \alpha_i y_i K(x, x_i) + b$ where x_i are support vectors



Application: kernel-based clustering

- Clustering (e.g., original K-Means) assumes linear separability in the original space
- Kernel tricks allow clustering of non-linearly separable data

- **Process:**

1. Uses a kernel function to compute distances or similarities in the transformed space.
2. Perform clustering (e.g., K-Means) in that space.

- But wait...

  Con: we don't have explicit high-dimensional points to calculate the mean



How to avoid explicit means

- Centroid in feature space: $\phi(m_k) = \frac{1}{|C_k|} \sum_{x_i \in C_k} \phi(x_i)$
- We don't compute $\phi(m_k)$ directly — too complex or infinite-dimensional

- **Instead:**
  - Use kernel values to measure distances to the "virtual" centroid.
  - All calculations stay in the original space using $K(x_i, x_j)$.
  - For two points x_i and x_j, we have

$$\begin{aligned} \|\phi(x_i) - \phi(x_j)\|^2 &= (\phi(x_i) - \phi(x_j))^\top (\phi(x_i) - \phi(x_j)) \\ &= \phi(x_i)^\top \phi(x_i) - 2\phi(x_i)^\top \phi(x_j) + \phi(x_j)^\top \phi(x_j) \\ &= K_{ii} - 2K_{ij} + K_{jj} \end{aligned}$$

- **Result:** Clusters form based on similarity, not explicit coordinates



Kernel K-Means algorithm

1. **Initialize**: Randomly assign points to k clusters.
2. **Assign**: For each point x_i, compute distance to each cluster's centroid:

$$\|\phi(x_i) - \phi(m_k)\|^2 = 1 - \frac{2}{|C_k|} \sum_{x_j \in C_k} K(x_i, x_j) + \frac{1}{|C_k|^2} \sum_{x_m, x_n \in C_k} K_{mn}$$

   Assign x_i to the closest cluster.
3. **Update**: Recompute kernel sums for each cluster based on new assignments.
4. **Repeat**: Until assignments stabilize.



Summary

1. **What is clustering?**
   - Unsupervised partitioning of data into groups based on similarity
   - No ground-truth labels during training
2. **Clustering evaluation metrics**
   - External: Purity, RI/ARI, FMI, NMI, V-measure
   - Internal: Silhouette, Calinski-Harabasz, Davies-Bouldin, Dunn
3. **Hard clustering methods**
   - k-Means: iterative assign–update, minimizes WCSS
   - Initialization matters: k-Means++, FFT, K-log K
   - k-Medoids: robust to outliers, uses actual data points as centers
   - Kernel k-Means: handles non-linearly separable clusters



What's next: L09 Clustering II

- Hierarchical clustering (agglomerative & divisive)
- Density-based clustering: DBSCAN
- Gaussian Mixture Models (GMM) and Expectation-Maximization (EM)
- Practical guidelines: choosing the right algorithm

Thank you! Questions?

